

debugging checklist for neural network training

1) "You didn't try to overfit a single batch first"

What this means

Before training on the full dataset, your model **must be able to perfectly memorize a tiny dataset** (even one batch).

If it can't, something is fundamentally wrong:

- model architecture
- loss
- labels
- optimizer
- data pipeline

Intuition

A neural net is a **universal function approximator**. Memorizing 32 samples should be trivial.

If it can't memorize:

| it definitely won't generalize

Example

```
x, y = next(iter(train_loader)) # one batch

for step in range(1000):
    optimizer.zero_grad()
    logits = model(x)
```

```
loss = criterion(logits, y)
loss.backward()
optimizer.step()
```

Expected behavior:

- loss → almost **0**
- accuracy → **100%**

If this doesn't happen:

- labels may be wrong
- shapes mismatched
- wrong loss
- bug in forward pass

This step isolates **learning mechanics** from **generalization**.

2) “You forgot to toggle train/eval mode”

What this means

Some layers behave **differently during training and inference**:

- Dropout
- BatchNorm

PyTorch requires you to **explicitly switch modes**.

Intuition

- `train()` → stochastic, regularized behavior
- `eval()` → deterministic, fixed behavior

If you forget this:

- validation accuracy becomes noisy or wrong
- BatchNorm statistics are incorrect

Example

```
model.train()  
# training loop  
  
model.eval()  
with torch.no_grad():  
    # validation loop
```

What goes wrong if you forget?

- Dropout keeps randomly zeroing units during validation
- BatchNorm keeps updating running stats on validation data

This silently ruins evaluation.

3) "You forgot to zero `.grad()` before `.backward()`"

What this means

In PyTorch, **gradients accumulate by default**.

Calling `.backward()` **adds** gradients instead of replacing them.

Intuition

Think of `.grad` as a bucket.

If you don't empty it, gradients pile up across steps.

That changes the optimization dynamics completely.

Example (WRONG)

```
loss.backward()  
optimizer.step()
```

Correct

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Symptom if forgotten

- loss explodes
- training unstable
- learning rate feels “wrong”

This mistake is so common it’s almost a rite of passage.

4) “You passed softmax outputs to a loss that expects raw logits”

This one is **extremely important**.

What this means

Many loss functions (like `CrossEntropyLoss`) already include:

- `LogSoftmax`
- `NLLLoss`

If you apply `softmax` yourself first, you **break the math**.

Intuition

Softmax squashes values into probabilities.

The loss wants **raw, unnormalized scores** (logits) so it can apply a **numerically stable softmax internally**.

Double-softmax = bad gradients.

Wrong

```
probs = torch.softmax(logits, dim=1)
loss = nn.CrossEntropyLoss()(probs, labels) # ❌
```

Correct

```
loss = nn.CrossEntropyLoss()(logits, labels) # ✅
```

Why this matters

- gradients become tiny
- learning slows or stalls
- model appears “stuck”

This mistake often looks like:

“My loss decreases very slowly for no reason”

The deeper pattern behind all these mistakes

All of them are about **violating assumptions**:

Mistake	Broken assumption
Not overfitting one batch	Model + loss can learn
Wrong train/eval mode	Layer behavior matches intent
Not zeroing grads	Gradients reflect current step
Softmax before loss	Loss receives correct inputs

Neural nets are brittle systems.

Small violations don’t crash — they **silently sabotage learning**.

A mental checklist for every new model

Before “real training”:

1. Can it overfit one batch?

2. Are `train()` / `eval()` used correctly?
3. Are gradients zeroed every step?
4. Am I passing logits (not probabilities) to the loss?

If all four are correct and learning still fails,
then you start tuning architecture and hyperparameters.

▼ a **minimal PyTorch training loop** annotated with these gotchas

```
import torch
import torch.nn as nn
import torch.optim as optim

# classification on toy data

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        # 2 layer net
        self.fc1 = nn.Linear(784,128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128,10)
        # outputs LOGITS (no softmax)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        return self.fc2(x) # <-- raw logits

model = Net()
loss = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-
```

3)

```
# TRAINING LOOP
model.train() # GOTCHA #2: enable training behavior (drop
out, batchnorm)

for epoch in range(num_epochs):
    for x, y in train_loader:

        # -----
        # Forward
        # -----
        logits = model(x)

        # GOTCHA #4 again: DO NOT softmax here
        loss = criterion(logits, y)

        # -----
        # Backward
        # -----
        optimizer.zero_grad() # GOTCHA #3: clear old gra
dients
        loss.backward()        # compute gradients
        optimizer.step()       # update weights

    print(f"epoch={epoch}, loss={loss.item():.4f}")

# Validation loop (also critical)

model.eval() # GOTCHA #2: switch to eval mode

correct = 0
total = 0
```

```

with torch.no_grad(): # no gradients during eval
    for x, y in val_loader:
        logits = model(x)

        # GOTCHA: no softmax needed for argmax
        preds = logits.argmax(dim=1)

        correct += (preds == y).sum().item()
        total += y.size(0)

accuracy = correct / total
print(f"val accuracy = {accuracy:.3f}")

# Inference / probability output

model.eval()
with torch.no_grad():
    logits = model(x)
    probs = torch.softmax(logits, dim=1) # NOW softmax is
    correct

```

Single-batch overfitting test (GOTCHA #1)

Run this **before** real training.

```

x, y = next(iter(train_loader))

model.train()
for step in range(500):
    logits = model(x)
    loss = criterion(logits, y)

```

```
optimizer.zero_grad()
loss.backward()
optimizer.step()if step %50 ==0:print(step, loss.item())
```

Expected:

- loss → ~0
- accuracy → 100%

If not:

- labels wrong
 - shapes wrong
 - loss mismatch
 - bug in forward pass
-

The four gotchas mapped explicitly

Gotcha	Where handled
Overfit one batch	single-batch test
train / eval mode	<code>model.train()</code> / <code>model.eval()</code>
zeroing gradients	<code>optimizer.zero_grad()</code>
logits vs softmax	loss definition + forward